IIT Madras — CSE Department

CS6530 – Applied Cryptography, Jul–Nov 2026

Assignment 1 Report

Secure Data Protection using AES-GCM and ChaCha20-Poly1305

Name: [**YOUR NAME**]
Roll Number: [**ROLL NUMBER**]
Submission Date: August 21, 2026

A secure data protection subsystem for generic chunked/packetized application records, using AES-128-GCM and ChaCha20-Poly1305 authenticated encryption, implemented in C++17 over OpenSSL's EVP API.

Contents

1 Introduction

This report documents the design, implementation, and validation of a secure data protection subsystem that exchanges application records between a `Sender` and a `Receiver` using Authenticated Encryption with Associated Data (AEAD). The subsystem supports two interchangeable AEAD configurations – **AES-128-GCM** and **ChaCha20-Poly1305** – both accessed through OpenSSL's EVP API, consistent with the assignment's emphasis on secure integration rather than primitive implementation.

The sender and receiver are assumed to already share a secret key (read from a key file at start-up); secure key establishment is explicitly out of scope. Network communication is likewise not required: the two roles exchange protected records through a shared file, `tm.bin`, which stands in for a transport channel.

All source code referenced in this report is included in the submission under `src/`. Build and run instructions are in the project `README.md`.

2 Design Summary

2.1 Architecture

The subsystem is organized into five classes, each responsible for exactly one concern:

Class	Responsibility	Location
`Record`	The application record: sequence id, timestamp, payload.	`record.h`
`Serializer`	`Record` ↔ fixed-width byte encoding.	`serializer.{h,cpp}`
`Crypt`	AEAD engine: nonce generation, encrypt/decrypt, wire-format framing, for both algorithms.	`crypt.{h,cpp}`
`Sender`	Build record → serialize → encrypt → write channel.	`sender.{h,cpp}`
`Receiver`	Read channel → decrypt → verify → replay-check → deserialize.	`receiver.{h,cpp}`

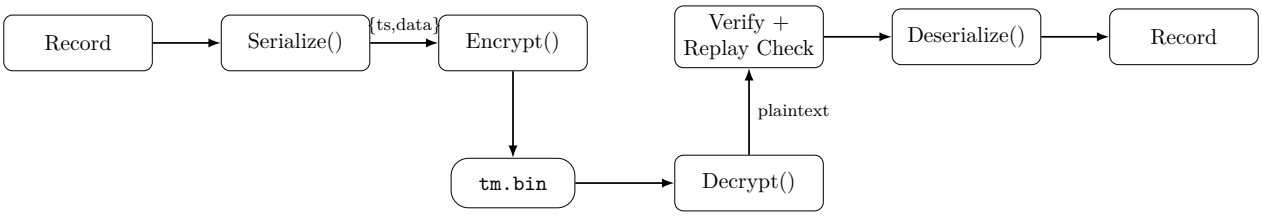

Figure 1: End-to-end record pipeline. `tm.bin` stores AAD (4B), nonce (12B), ciphertext, and tag (16B) concatenated.

2.2 Wire Format

Each protected record written to `tm.bin` is a single concatenation of four fixed-position fields:

$$\texttt{secureStream} = \underbrace{\texttt{AAD}}_{4\text{ B}} \,\|\, \underbrace{\texttt{nonce}}_{12\text{ B}} \,\|\, \texttt{ciphertext} \,\|\, \underbrace{\texttt{tag}}_{16\text{ B}}$$

Because every field except the ciphertext has a length fixed by the algorithm choice, the receiver can slice the blob back apart by fixed offsets without a length-prefixed or self-describing envelope.

2.3 Nonce Management (FR-5 / SR-3)

A fresh 96-bit nonce is generated via OpenSSL's `RAND_bytes` (a CSPRNG) for *every* call to `Crypt::Encrypt`; no nonce is ever cached, derived from a counter, or reused. This is the standard nonce strategy for both GCM and ChaCha20-Poly1305, and needs no persisted state across sender restarts – the trade-off is a birthday-bound collision risk that only becomes material after roughly 2^{32} messages under one key, far beyond this subsystem's demonstrated volume (see TR-7, Section 4.7).

2.4 Associated Data Selection (FR-4 / SR-4)

The record's 4-byte sequence id is authenticated as AAD. It remains visible in plaintext on the wire (sequence ids are not secret) but cannot be detached from its ciphertext, or swapped for a different record's id, without invalidating the authentication tag. This binding is what makes replay detection possible: the receiver only trusts a record's id after the tag has verified.

2.5 Replay Handling Strategy (FR-8 / SR-5)

The `Receiver` maintains an in-memory set, `seenRecIds`, of every record id it has accepted in the current session. After a record decrypts and its authentication tag verifies, its id (recovered from the now-trusted AAD) is checked against this set; a repeat is rejected before `Receive()` ever returns data to the caller. This strategy accepts records in any order (not just strictly increasing ids) and detects a genuine replay unambiguously. Its main limitation – discussed further in Section 6 – is that the set grows without bound for the life of the session, which a production system would instead bound with a sliding window (as TLS and IPsec do).

2.6 Authentication Failure Handling (FR-7 / SR-6)

OpenSSL's decrypt API only confirms authenticity at `EVP_DecryptFinal_ex`, which returns ≤ 0 on a tag mismatch; `EVP_DecryptUpdate` itself will produce plaintext-shaped bytes even for a corrupted ciphertext. `Crypt::Decrypt` treats the final-step return value as the single point of truth and *throws* on failure rather than returning a status flag, so a failure can only be silently ignored by an explicit, conscious `catch` – which is exactly the fail-closed behaviour `Receiver::ConstructRecord` implements: any exception during decryption, verification, or replay checking results in `Receive()` returning an empty string, with nothing recovered ever exposed to the caller.

2.7 Key Management (Out of Scope)

Per the assignment's stated assumptions, the sender and receiver already share a secret key; this subsystem reads that key from a binary file (`keys/aeskey.bin`, 16 bytes for AES-128-GCM; `keys/ccpkey.bin`, 32 bytes for ChaCha20-Poly1305). No key exchange, rotation, or derivation is implemented, in line with the assignment's Section 3.2 scope exclusions.

3 Testing Methodology

The assignment specifies three logical entities: a **Sender** that generates valid protected records, a **Malicious Actor** that generates or modifies protected records to simulate invalid conditions, and a **Receiver** that validates and processes received records. All three are realized as logical components within a single test program, `src/test_suite.cpp`, rather than as separate communicating processes – an option the assignment explicitly permits (*"network communication is optional … provided that all Functional Requirements and Testing Requirements are satisfied"*).

The malicious actor is implemented as a function, `CorruptByteAt(offset)`, that opens `tm.bin` after a legitimate `Send()` and flips a single byte via XOR at a computed offset – inside the ciphertext,

the tag, or the AAD depending on the test. This achieves the same adversary model as a network-level attacker who can modify bytes in transit, without the added complexity of standing up sockets or processes.

Every Testing Requirement below (TR-1 through TR-6) is executed once for AES-128-GCM and once for ChaCha20-Poly1305, using independent 16-byte and 32-byte keys respectively, as required by the assignment.

4 Testing Results

4.1 TR-1 – Positive Baseline Test

Objective Demonstrate successful protection, transmission, verification, and recovery of valid application records.

Procedure Construct a `Sender` and `Receiver` sharing the same key and AEAD configuration. Call `Sender::Send(msg)`, then `Receiver::Receive()`, and compare the recovered string to `msg`.

Test Input
`"TR-1 baseline message for AES-128-GCM"` / `"...for ChaCha20-Poly1305"`

Expected Behaviour
The recovered string exactly equals the original input.

Observed Behaviour
Recovered string matched the input for both configurations.

Outcome **PASS** (AES-128-GCM), **PASS** (ChaCha20-Poly1305)

```
[PASS] TR-1 - Positive baseline (AES-128-GCM)
[PASS] TR-1 - Positive baseline (ChaCha20-Poly1305)
```

4.2 TR-2 – Ciphertext Integrity Test

Objective Demonstrate that modification of the protected record's ciphertext causes authentication failure and rejection by the receiver.

Procedure Send a valid record, then have the malicious actor flip one byte at offset 16 of `tm.bin` – the first byte of the ciphertext (immediately after the 4-byte AAD and 12-byte nonce) – before calling `Receive()`.

Test Input
`"TR-2 ciphertext tamper test"`, tampered at byte offset 16.

Expected Behaviour
`Receive()` returns an empty string; no plaintext is released.

Observed Behaviour
`Receive()` returned `""` for both configurations (verified via `received.empty()`).

Outcome **PASS** (AES-128-GCM), **PASS** (ChaCha20-Poly1305)

```
[PASS] TR-2 - Ciphertext integrity (AES-128-GCM)
[PASS] TR-2 - Ciphertext integrity (ChaCha20-Poly1305)
```

4.3 TR-3 – Authentication Tag Test

Objective Demonstrate that modification of the authentication tag causes verification failure and rejection.

Procedure Send a valid record, then flip the final byte of `tm.bin` – which always falls within the trailing 16-byte tag regardless of payload length – before calling `Receive()`.

Test Input
`"TR-3 tag tamper test"`, tampered at the last byte.

Expected Behaviour
`Receive()` returns an empty string.

Observed Behaviour
`Receive()` returned `""` for both configurations.

Outcome **PASS** (AES-128-GCM), **PASS** (ChaCha20-Poly1305)

```
[PASS] TR-3 - Authentication tag (AES-128-GCM)
[PASS] TR-3 - Authentication tag (ChaCha20-Poly1305)
```

4.4 TR-4 – Associated Data (AAD) Test

Objective Demonstrate that modification of the Associated Data causes verification failure and rejection.

Procedure Send a valid record, then flip byte offset 0 of `tm.bin` – inside the 4-byte AAD (record id) – before calling `Receive()`.

Test Input
`"TR-4 AAD tamper test"`, tampered at byte offset 0.

Expected Behaviour
`Receive()` returns an empty string.

Observed Behaviour
`Receive()` returned `""` for both configurations.

Outcome **PASS** (AES-128-GCM), **PASS** (ChaCha20-Poly1305)

```
[PASS] TR-4 - AAD integrity (AES-128-GCM)
[PASS] TR-4 - AAD integrity (ChaCha20-Poly1305)
```

4.5 TR-5 – Replay Test

Objective Demonstrate that replay of a previously accepted protected record is detected and rejected.

Procedure Send one valid record and call `Receive()` once (expected to succeed). Without any further `Send()`, call `Receive()` again on the same, unchanged `tm.bin` – simulating an attacker resubmitting a captured record.

Test Input
`"TR-5 replay test"`, received twice.

Expected Behaviour
The first `Receive()` returns the original message; the second returns an empty string.

Observed Behaviour
First call succeeded and matched the input; second call returned `""` for both configurations (rejected via the `seenRecIds` check in `Receiver::ConstructRecord`).

Outcome **PASS** (AES-128-GCM), **PASS** (ChaCha20-Poly1305)

```
[PASS] TR-5 - Replay handling (AES-128-GCM)
[PASS] TR-5 - Replay handling (ChaCha20-Poly1305)
```

4.6 TR-6 – Wrong-Key Test

Objective Demonstrate that use of an incorrect cryptographic key causes verification failure and rejection.

Procedure Send a record encrypted under one key; construct the `Receiver` with a different key of the same length (16 bytes for AES-128-GCM, 32 for ChaCha20-Poly1305) and call `Receive()`.

Test Input
`"TR-6 wrong key test"`, decrypted with a mismatched key.

Expected Behaviour
`Receive()` returns an empty string.

Observed Behaviour
`Receive()` returned `""` for both configurations.

Outcome **PASS** (AES-128-GCM), **PASS** (ChaCha20-Poly1305)

```
[PASS] TR-6 - Wrong-key rejection (AES-128-GCM)
[PASS] TR-6 - Wrong-key rejection (ChaCha20-Poly1305)
```

4.7 TR-7 – Nonce Management Verification

Objective Demonstrate that the nonce management approach (Section 2.3) prevents nonce reuse under normal operation, per the assignment's guideline of processing approximately 10,000 records under one key.

Procedure Using a single `Sender` instance and a fixed key, send 10,000 records in sequence. After each `Send()`, read the 12-byte nonce field back out of `tm.bin` (offset 4, immediately after the AAD) and insert it into a `std::set`. A collision would shrink the set below the message count.

Test Input
10,000 sequential records (`"nonce-test-record-0"` …`"nonce-test-record-9999"`), one key per algorithm.

Expected Behaviour
The set of observed nonces contains exactly 10,000 unique entries – zero collisions.

Observed Behaviour
10,000 unique nonces observed for both configurations, with no duplicate detected at any of the 1,000-record progress checkpoints.

Outcome **PASS** (AES-128-GCM), **PASS** (ChaCha20-Poly1305)

```
--- TR-7: Nonce Management Verification ---
No reuse detected till 1000th iteration
No reuse detected till 2000th iteration
No reuse detected till 3000th iteration
No reuse detected till 4000th iteration
No reuse detected till 5000th iteration
No reuse detected till 6000th iteration
No reuse detected till 7000th iteration
No reuse detected till 8000th iteration
No reuse detected till 9000th iteration
[PASS] TR-7 - Nonce uniqueness over 10000 records (AES-128-GCM) (10000 unique nonces observed)
No reuse detected till 1000th iteration
...
No reuse detected till 9000th iteration
[PASS] TR-7 - Nonce uniqueness over 10000 records (ChaCha20-Poly1305) (10000 unique nonces observed)
```

Correctness Summary

Requirement	AES-128-GCM	ChaCha20-Poly1305	Total
TR-1 – Positive baseline	**PASS**	**PASS**	2/2
TR-2 – Ciphertext integrity	**PASS**	**PASS**	2/2
TR-3 – Authentication tag	**PASS**	**PASS**	2/2
TR-4 – AAD integrity	**PASS**	**PASS**	2/2
TR-5 – Replay handling	**PASS**	**PASS**	2/2
TR-6 – Wrong-key rejection	**PASS**	**PASS**	2/2
TR-7 – Nonce uniqueness (10,000 msgs)	**PASS**	**PASS**	2/2
Total	**7/7**	**7/7**	**14/14**

5 Performance Analysis (TR-8)

Objective and Procedure

Compare AES-128-GCM and ChaCha20-Poly1305 by measuring encrypt/decrypt cost at three payload sizes – 64 B, 1 KiB, and 64 KiB – as required by the assignment. Timing is taken directly around `Crypt::Encrypt/Decrypt` (bypassing `Sender/Receiver`'s file I/O), since disk I/O time would otherwise dominate and mask the actual cost difference between the two ciphers; this is a deliberate scoping decision, not an oversight. Each measurement is averaged over 2,000 iterations (500 for 64 KiB, to keep total runtime reasonable), using `std::chrono::high_resolution_clock`. All measurements were taken on an Intel Core i5-12450H (12th Gen), which exposes the `AES-NI` and `PCLMULQDQ` instruction extensions.

Observed Results

Algorithm	Size	Encrypt (μs)	Decrypt (μs)	Enc MB/s	Dec MB/s
AES-128-GCM	64 B	2.27	1.21	26.95	50.57
ChaCha20-Poly1305	64 B	2.14	1.29	28.49	47.41
AES-128-GCM	1 KiB	2.82	1.36	346.08	715.91
ChaCha20-Poly1305	1 KiB	3.38	2.00	288.98	487.11
AES-128-GCM	64 KiB	88.67	30.38	704.88	2057.10
ChaCha20-Poly1305	64 KiB	112.55	81.05	555.31	771.15

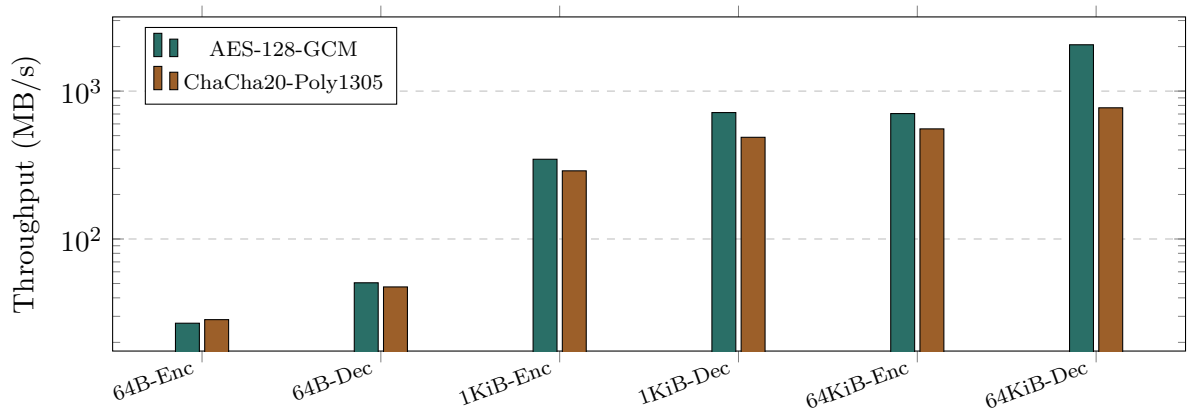

Figure 2: Throughput by payload size and operation (log scale).

Discussion

At 64 B, the two algorithms are within noise of each other – fixed per-call overhead (EVP context setup) dominates at that size rather than the cipher's steady-state throughput. From 1 KiB onward, AES-128-GCM pulls ahead by roughly 1.2–2×. This is directly attributable to hardware acceleration: this CPU's `AES-NI` instruction computes one AES round per instruction, and `PCLMULQDQ` similarly accelerates GCM's GHASH computation, while ChaCha20-Poly1305 has no equivalent hardware fast path and executes as general-purpose ARX/polynomial arithmetic on the ALU regardless of the underlying CPU. On hardware *without* AES-NI – common on low-end ARM cores and some embedded/IoT targets – this result would invert, which is precisely why ChaCha20-Poly1305 is standardized as a TLS 1.3 cipher suite alongside AES-GCM rather than superseded by it.

6 Discussion

Implementation Assumptions

- The sender and receiver share a pre-provisioned key, read from a local file; no key exchange or rotation is implemented (assignment Section 3.2).
- `tm.bin` is treated purely as a byte-transport abstraction; no access control on the file itself is assumed necessary, since the AEAD tag already defends against undetected tampering by anyone able to write to it.
- The replay-protection set (`seenRecIds`) is unbounded and scoped to a single `Receiver` instance's lifetime – adequate for this assignment's demonstrated volume (up to 10,000 records per TR-7), but a long-running service would need a bounded sliding window instead (as used by TLS record replay windows and IPsec ESP anti-replay windows) to avoid unbounded memory growth.
- TR-8 performance is measured on the `Crypt` layer directly, bypassing file I/O, to isolate cryptographic cost from disk latency.

Configuration Equivalence (FR-9)

Both AEAD configurations are exercised through the identical `Sender`/ `Receiver`/`Crypt` interface and pass an identical set of TR-1 through TR-7 checks; the only externally visible difference is the required key length (16 bytes vs. 32 bytes) and, as shown in Section 5, runtime performance.

Limitations

- No key rotation or expiry mechanism.
- No transport-level authentication of `tm.bin` itself, beyond what the AEAD tag already provides for the payload.
- `Sender::sendCount` is a single process-wide counter; a multi-sender deployment sharing one key would additionally need a partitioned nonce/id scheme (a fixed sender-id prefix plus a local counter, as used by QUIC and IPsec Security Associations) to keep id/nonce spaces disjoint across senders.

None of these are deficiencies relative to the assignment's stated scope – they are documented boundaries of what was intentionally left out, consistent with Section 3.2 of the assignment brief.

7 Conclusion

The implemented subsystem satisfies all stated Functional and Security Requirements: both AEAD configurations correctly protect and recover application records (FR-1–FR-3, FR-9), Associated Data and authentication tags are verified before any plaintext is released (FR-4, FR-6, FR-7, SR-1, SR-2, SR-4, SR-6), nonces are generated freshly per message with empirically zero collisions across 10,000 records per algorithm (FR-5, SR-3), and replayed records are detected and rejected (FR-8, SR-5). All 14 correctness checks (TR-1–TR-7, across both configurations) pass, and the performance comparison (TR-8) shows AES-128-GCM outperforming ChaCha20-Poly1305 by roughly 1.2–2× on this AES-NI-equipped machine, consistent with the expected behaviour of a hardware-accelerated versus software-only AEAD construction.